Native Material Profiles and Qualified Graph Readouts

Claims and presentations. Unqualified `set:` claims use the proved `prime-common` material fragment. Additional logical, foundation, choice and universe laws require an explicit profile or declaration. A graph presentation is data, not an adoption of its interpretation's complete set theory. Profiles and readouts are separate: the same stored presentation can be inspected under several qualified readouts without changing its authored data.

The profile-indexed first-order source signature contains equality and membership. A derivation retains its declarations, hypothesis occurrences and law origins. Checked interpretations connect these derivations to the constructed contextual graph model and to separately qualified material models. The profile catalogue keeps four statuses separate: declared theory, constructed model, checked interpretation and executable fragment. A literature result, a finite picture or a native replay is not promoted to a proof of a full native set theory. Catalogue entries with different signatures are separate theories. A comparison with the common material fragment requires a proved interpretation; catalogue membership does not make an entry a stronger extension of that fragment.

Finite values. A source `(graph r (row`$_0$`...row`$_{n-1}$`))` has a valid root and valid successor indices. Rows retain order and repeated occurrence positions. The adjacency relation uses the support of those rows: a repeated index is one edge, while two distinct indices remain two target vertices until the selected material interpretation identifies them.

Readout	Identity identified	Boundary
`presentation`	Authored graph data, including row occurrences	Structural equality
`aczel`	Greatest bisimulation; equality of the Aczel decorations	Material membership preserved
`raw-unfolding`	Isomorphism of full rooted unfolding trees	Distinct adjacency targets are counted
`scott-canonical`	Equality of projected nodes after iterated unfolding quotient and edge deduplication	Final graph is Scott-extensional
`foundation`	Aczel identity restricted to accessible roots	Partial, membership-preserving reading into well-founded sets
`observed`	Unlabelled graph behavior, outcome, scope and origin ledger	Results and faults remain distinct

The canonical finite Scott construction computes actual quotient carriers, surjective projections and member images. Every genuine merge decreases the carrier size; the input size bounds the number of quotient rounds. Raw unfolding comparison must be repeated after removing duplicate

quotient edges. For example, two empty child vertices and one empty child vertex initially have different raw trees but the same canonical material reading. Transport to another carrier requires a canonical decoration of the final graph. Ordinary decorations alone do not determine this identity kernel. Closed embeddings preserve complete successor fibres and induce closed embeddings through the quotient iterations. Consequently a comparison in a disjoint joint graph agrees with full unfolding isomorphism of the independently canonical roots. Equality is reflexive, symmetric and transitive, and adding unrelated graph components does not change it. Membership agrees with equality to one of the parent's selected child representatives. An injective edge-preserving map that introduces new successors does not satisfy this preservation contract. This finite construction does not supply a complete Scott anti-foundation universe.

The Foundation reading checks accessibility of the root, rather than absence of every cycle in the stored graph. An unreachable cycle is harmless to that root; a reachable cycle is outside the readout's domain. No total operation silently sends a cyclic set to the empty set and then claims to preserve membership.

Evidence consumers. Material support may identify different presentations or different child occurrences. A consumer using membership support can descend through that quotient when its family and term are compatible with the proved kernel and context transport. A consumer inspecting origin positions or provenance may require the retained presentation instead. Compatible dependent sections and their substitution comparisons are proved in the contextual model; bare equality of displayed values does not establish those contracts.

An observed source is `(observed-graph G outcome scope origins)`, where the outcome is `(returned k)` or `(faulted k)`. Its origin receipt retains the complete authored source, an in-range index and the corresponding origin. Equality of observed readings permits transport of the appropriate index and origin value; it does not replace the retained source by another presentation. A proof-path count is a separate observation and is not a count of independent evidence. The finite observed surface does not itself encode labelled execution histories or every future operational observation. Those require their declared source semantics and the corresponding richer interpretation.

Proof compilation and authority. The general source compiler constructs retained typed HOL proof syntax. Its codec and decoder check object-variable indices, hypothesis positions, types and conclusions. Recovery is exact for the retained HOL target proof tree, including its annotations and hypothesis occurrences. The first-order compiler interprets weakening through an occurrence map and expands derived connectives; its injection on the original source trees is not asserted. The typed receipt envelope separately retains declarations, adoptions, origins, source context and variable interpretation. Bytes alone do not recover opaque primitive adoptions or authorize them. The native code renderer is a separate boundary. Structural logical equality uses the explicit `hol-equality` rule bundle; transport-only `eq-subst` does not imply reflexivity. Native equality proofs lower to dependent reflexivity and transport terms, which the regular kernel checks in their complete admitted context. Neither a type annotation nor serialized data adopts an assumption. Withdrawing a required declaration invalidates reuse of a dependent theorem or saved native package.

Executable surface and qualification.

```
(set:interpret finite-graph aczel (equal G H))
(set:interpret finite-graph aczel (member G H))
(set:interpret finite-graph scott-canonical (members G))
(set:interpret finite-graph foundation (eligible G))
```

```
(set:interpret finite-graph observed (origin-at 0 i))
```

Equality and membership always name a readout. Returned finite values retain their original presentations; ordinary atom equality is not material equality. A list of material members returns first authored representatives, not a canonical printing order. Known malformed closed input is refuted; open input or a root outside the Foundation domain is undetermined. Exhausted work or storage is incomplete. Unknown interpreters remain data.

General solver, interpretation, substitution and recovery theorems are kernel evidence. Native execution is qualified separately against checked solver output, source-proof exports and independent controls, with source, artifact and binary hashes retained. This establishes the stated replay agreement, not a proof of the C implementation's correctness for all inputs. Native universal construction, larger model comparisons and unrestricted library transport retain their own obligations.